

Deep Learning

Assignment #4

Transformers

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 15 points*
- *Timeline: Assignment #4 available in Week 13 (25th Nov), Code submission deadline till Week 15 (9th Dec), offline defense deadline till Week 15 (9th Dec)*

While completing this assignment, you will develop and strengthen the following skills:

- *Approaching a real-world machine learning problem with given data and a target metric.*
- *Conducting effective Exploratory Data Analysis (EDA).*
- *Designing a reliable validation strategy and monitoring it throughout the entire development process.*
- *Building a Transformer-based model from scratch or using existing architectures.*
- *Performing health checks on the optimization process.*
- *Using Transfer Learning techniques and other advanced Deep Learning techniques*
- *Utilizing specialized Deep Learning frameworks and logging tools.*
- *Creating a comprehensive performance report.*

Task

Task Selection:

- *Take part in the [dedicated Kaggle In-Class competition](#)*

EDA and Metric Analysis:

- *Perform Exploratory Data Analysis (EDA).*
- *Analyze the proposed target metric: discuss its strengths, weaknesses, and potential edge cases.*
- *Suggest complementary metrics to capture other aspects of performance.*
- *Propose an alternative target metric, with justification.*

Validation:

- Design a validation strategy based on common sense and expected correlation with Public leaderboard scores.
- Explain your motivation for the chosen strategy.
- Discuss possible reasons for low or high correlation with leaderboard results.
- Perform adversarial validation if necessary (make sure to do it with the same model as the main task)

Transformer Encoder model:

- Set up experiment and logging infrastructure using high-level libraries (Huggingface transformers and W&B). It's much better to use Huggingface transformers and not Lightning here.
- Use existing architecture based on Transformer Encoder to solve Named Entity Recognition task.
- Experiment with different BERT-like multilingual encoders: e.g. multilingual BERT, XLM-RoBERTa, mDeBERTa-v3, ruRoberta-large, EuroBERT. Be sure that model of your choice actually supports ukrainian and other slavic or mixed-code in slavic-like languages.
- Experiment with different Optimizers, Schedulers, hyperparameters, and all other elements, which were covered in the Lecture materials
- Try threshold optimization and different entity-merging algorithms.
- Collect all relevant results: loss/metric curves, gradient flow visualizations, parameter tuning logs, etc.
- Submit predictions and compare Public leaderboard metrics with local validation results.
- Conclude whether the model is adequate.

Ensembling:

- Combine several encoder models using the ensembling (NOT stacking) technique, maybe try to ensemble with Lab3 RNN-like models.
- Submit predictions to obtain Public leaderboard metrics, and compare them with local validation results.

Final Report

- Prepare a comprehensive report including:
 - data insights,
 - feature engineering,
 - tried models,
 - chosen hyperparameters,
 - metrics and optimization curves,
 - model insights and conclusions.
- Reports may be prepared in LaTeX or Markdown ([README.md](#)).

NOTES

- Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a “lucky shot.” Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself, and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.
- Keep your EDA concise. Avoid overloading it with excessive plots, histograms, or tables. Focus only on key aspects of the data (e.g., missing values, target distribution). If a part of your EDA does not lead to meaningful conclusions, you likely don't need it (for example, full pairwise correlation matrices). Do NOT follow the typical Kaggle-style EDA – most of them are far too verbose and unfocused.
- Prepare your presentation carefully. A messy speech with too much code or too many files makes it nearly impossible for the audience to follow. Think in advance about what you want to highlight
- Using Keras, TensorFlow, or Chainer is a RED FLAG and will result in an instant BAN!
- You should thoroughly analyze the training/test split and consider time and group factors in your validation. You should also use the same splits for different ML/DL approaches so that the results can be accurately compared. And if there is a huge mismatch between CV vs kaggle LB - consider using adversarial validation.
- Your final report MUST include both CV and LB scores for all experiments.

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	Points
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	8
Practical question about the submitted code during defense	2
Theoretical part during defense (2 questions + 1 exercise)	5
Criterion	Points

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the

Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 2 and 1 Exercise from the Appendix 3, but will not necessarily be restricted to them.

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.
- Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.
- Language assistance tools: Grammarly, DeepL, Google Translate, etc.
- Local LLMs or self-hosted AI models.

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.
- Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.
- Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.
- Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.
- Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.
- Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.

Conditions for Use

Full Transparency:

- Any use of AI tools must be disclosed in your submission.
- Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.
- If using online tools, include chat links (if available) or full transcripts in an appendix of your report.

Partial Contribution Only:

- AI tools should support your work — they must not produce the complete assignment, lab, or report.
- You must be able to explain and defend any AI-generated content during oral defense or Q&A

Examples

Acceptable Uses:

- "Correct grammar and style in this paragraph of my report."
- "Write a function to load images from a directory and resize them."
- "Suggest 3 ideas for improving model generalization."

Unacceptable Uses:

- "Write the entire peer review for this paper."
- "Generate the complete solution code for this assignment task."

Appendix 1 (Theoretical Questions)

- What happens if we don't use positional encoding in transformers? Will the metrics degrade significantly if we take a pretrained model, remove positional encoding and try to fine-tune the model for our task (e.g., sentence classification) without positional encoding?
- Do we have a cross-attention mechanism in BERT architecture?
- How does multi-head attention differ from single head attention, and why might it be beneficial?
- Describe the two main pretraining tasks BERT uses during its unsupervised training. Can we use simple text classification or token classifications tasks for the pretraining process in BERT-like models and what are pros and cons of such pretraining?
- We have 2 BERT-based models: 1 - text classification for binary classification (dim of output linear = 2) 2 - token classification for task with a single meaningful class (dim of output linear = 2) A) Is the number of trainable parameters different in these two models? B) Is the time required for train/inference different in these two models on the same hardware?
- Can you explain the fundamental difference between generative and discriminative modeling in the context of machine learning? How do these two approaches diverge in their objectives, and in what types of tasks or applications is one more suitable than the other? Provide examples for NLP
- Can you provide and explain different ways of generating tokens from a decoder output for text generation models? Maybe you can come up with your own ideas.
- Explain the differences between encoder-only, decoder-only, and encoder-decoder models.
- What is the difference between the output generation process during training and during inference in encoder-decoder models?
- Why do we handle output generation differently during training and inference in encoder-decoder models?
- Explain how to construct a vocabulary for a word-level tokenizer and for a BPE tokenizer.
- Explain how transformers are applied to image processing using ViT-style patchification.
- Explain how transformers are applied to audio data.
- Which of the metrics are common in generative NLP e.g. in translation or summarization? And what are the pros and cons of these metrics?
- Explain what distillation is. When should we use it, and what are its main advantages and disadvantages?
- Can knowledge distillation be effectively combined with semi-supervised learning for tasks like text classification or sequence labeling? For instance, if we generate pseudo-labels from a teacher model on unlabeled data, how should we balance the distillation loss with the supervised loss? What are the key risks of this approach (e.g., confirmation bias), and how can they be mitigated?
- Explain how to correctly perform multi-stage model stacking. What are the main challenges in the validation process? Which methods can be used to mitigate these issues? Add formulas to formally illustrate the stacking procedure.
- What are the main objectives in a Kaggle competition? How does this differ from industrial machine-learning tasks and academic research? What are the main advantages and disadvantages of the Kaggle-style approach to solving ML problems?

Appendix 2 (Practical Exercises)

1. **Self-attention: small forward pass (single head).** Consider a single-head self-attention layer without scaling ($1/\sqrt{d}$ factor omitted). For a sequence of two 2D input tokens $x_1, x_2 \in \mathbb{R}^2$ we define

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad X = \begin{bmatrix} x_1^\top \\ x_2^\top \end{bmatrix},$$

and the attention weights and outputs as

$$s_{ij} = q_i^\top k_j, \quad \alpha_{ij} = \frac{e^{s_{ij}}}{\sum_{m=1}^2 e^{s_{im}}}, \quad o_i = \sum_{j=1}^2 \alpha_{ij} v_j.$$

Let

$$x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad W_Q = I_2, \quad W_K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad W_V = I_2.$$

- (a) Compute the query, key and value vectors $q_1, q_2, k_1, k_2, v_1, v_2$.
 - (b) Compute the score matrix $S = [s_{ij}]_{i,j=1}^2$.
 - (c) Compute the attention weights α_{ij} exactly (you may leave expressions in terms of e such as $\frac{e}{e+1}$).
 - (d) Compute the output vectors o_1 and o_2 .
2. **Self-attention: properties of attention weights.** Scaled dot-product self-attention for a fixed query q_i and keys $\{k_j\}_{j=1}^T$ defines

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}, \quad \alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{m=1}^T \exp(s_{im})}.$$

- (a) Show that for each query position i , the attention weights form a probability distribution over keys, i.e.

$$\alpha_{ij} \geq 0 \quad \text{for all } j, \quad \sum_{j=1}^T \alpha_{ij} = 1.$$

- (b) Let $v_1, \dots, v_T \in \mathbb{R}^d$ be the value vectors. Show that the attention output

$$o_i = \sum_{j=1}^T \alpha_{ij} v_j$$

is a convex combination of the values. Conclude that

$$\|o_i\|_2 \leq \max_{1 \leq j \leq T} \|v_j\|_2.$$

3. **Encoder–decoder cross-attention: small forward pass.** In an encoder–decoder transformer, cross-attention uses queries from the decoder and keys/values from the encoder outputs.

Consider an encoder that produces two 2D hidden states

$$e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad e_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

and a decoder representation (e.g., after its masked self-attention layer)

$$d = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Cross-attention is defined as

$$q = W_Q^{(\text{dec})} d, \quad K = E W_K^{(\text{enc})}, \quad V = E W_V^{(\text{enc})}, \quad E = \begin{bmatrix} e_1^\top \\ e_2^\top \end{bmatrix},$$

where $q \in \mathbb{R}^2$ is the query (from the decoder), while $K \in \mathbb{R}^{2 \times 2}$ and $V \in \mathbb{R}^{2 \times 2}$ are keys and values (from the encoder outputs). Let the attention scores (no scaling) be

$$s_j = q^\top k_j, \quad j = 1, 2,$$

attention weights

$$\beta_j = \frac{e^{s_j}}{e^{s_1} + e^{s_2}},$$

and context

$$c = \beta_1 v_1 + \beta_2 v_2,$$

where $k_j^\top$ and $v_j^\top$ are the j -th rows of K and V respectively.

Let

$$W_Q^{(\text{dec})} = I_2, \quad W_K^{(\text{enc})} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad W_V^{(\text{enc})} = I_2.$$

- Compute q , k_1 , k_2 , v_1 , v_2 .
- Compute the scores s_1, s_2 and attention weights β_1, β_2 (you may leave them in terms of e).
- Compute the context vector c .
- Briefly explain which objects in this computation come from the decoder (queries) and which come from the encoder (keys and values), and how this allows the decoder to focus on relevant encoder positions.

4. **Vision Transformer: patchification and embedding.** Consider a grayscale image of size 4×4 pixels, represented as

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 2 & 2 & 0 & 0 \\ 0 & 0 & 3 & 3 \end{bmatrix}.$$

We will apply ViT-style patchification with patch size $P = 2$, so each patch is 2×2 and is flattened into a vector in $\mathbb{R}^4$. The patch embedding is a linear map $W_{\text{patch}} \in \mathbb{R}^{2 \times 4}$:

$$W_{\text{patch}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}.$$

- (a) Draw the 2×2 grid of non-overlapping patches for this 4×4 image and write down the four flattened patch vectors $p_1, p_2, p_3, p_4 \in \mathbb{R}^4$ (row-major order inside each patch).
- (b) Compute the embedded patch tokens $z_i = W_{\text{patch}} p_i \in \mathbb{R}^2$ for $i = 1, \dots, 4$.